

# When FLOPs Mislead on PC GPUs: RTX 4060 Inference Energy and Batch-Size Effects

Aryan Shah

Texas Academy of Mathematics and Science  
University of North Texas  
Denton, TX, USA  
aryanshah.2431@gmail.com

Romir Kadiam

Texas Academy of Mathematics and Science  
University of North Texas  
Denton, TX, USA  
romirkadiam123@gmail.com

**Abstract**—This paper begins a separate PC-focused measurement study for neural-network inference energy. The first system is a Windows desktop with an NVIDIA GeForce RTX 4060 running CUDA PyTorch inference on five image-classification models. Each model is measured for ten 10-second windows with randomized order and raw power logs retained. The run measures CUDA latency and GPU-only board energy estimated from `nvidia-smi`; it does not measure CPU package energy or whole-system wall power. On this first PC system, latency explains most between-model GPU-energy variation ( $R^2 = 0.963$ ), while FLOPs is much weaker ( $R^2 = 0.038$ ). We also run a batch-size sweep on the same GPU; batching reduces per-image GPU energy by  $3.3\times$  to  $18.6\times$  depending on the model, showing that batch size is a first-order experimental control for GPU energy claims. The result motivates expanding the PC benchmark across additional Windows and non-Apple systems while keeping it separate from the Apple-Silicon CPU-only paper.

**Index Terms**—GPU inference, energy measurement, RTX 4060, Windows, PyTorch, benchmarking

## I. Introduction

This document is intentionally separate from the Apple-Silicon inference-energy paper. The Apple paper studies CPU-only PyTorch inference on MacBook hardware using powermetrics. This PC paper studies non-Mac systems, beginning with Windows CUDA inference on an RTX 4060 and intended to expand to additional test machines.

The goal is not to compare absolute joules across MacBook and PC runs. The power boundaries are different: the Apple measurements use package-level powermetrics data, while the RTX 4060 run uses GPU board-power samples reported by `nvidia-smi`. Instead, this paper asks a local systems question: under a fixed PC runtime, do common architecture proxies such as FLOPs track measured GPU inference energy?

The current contributions are:

- A reproducible Windows CUDA measurement workflow for five common image-classification architectures, including raw `nvidia-smi` logs, environment metadata, randomized model order, and confidence intervals.
- A first RTX 4060 dataset showing that latency is a much stronger predictor of GPU-only inference energy than FLOPs under this runtime.

- A batch-size sweep showing that per-image GPU energy changes by up to  $18.6\times$  relative to batch size 1 and that the energy-optimal batch size is model-dependent.
- A clear power-boundary statement: the current artifact measures GPU board-power estimates only, not CPU package energy or wall-plug energy.

## II. Related Work and Motivation

Efficient model design often reports FLOPs, parameter count, or model size because these metrics are portable and easy to compute before deployment. Architectures such as MobileNet, EfficientNet, and residual networks make different tradeoffs among depthwise convolution, channel width, residual connectivity, and operator mix [1]–[3]. These architecture-level metrics do not directly measure kernel launch overhead, memory traffic, tensor-core utilization, or batching behavior on a deployed GPU.

Systems benchmarking work has repeatedly emphasized measured deployment behavior over abstract operation counts. MLPerf Inference, for example, separates latency-oriented and throughput-oriented scenarios rather than assuming one number characterizes all deployments [4]. NVIDIA’s `nvidia-smi` and NVML tooling expose GPU telemetry useful for local power studies, but the boundary is the GPU device rather than whole-system energy [5]. This paper follows that measurement-first approach for commodity PC GPUs.

## III. Measurement Scope

Table I summarizes the first PC measurement environment. The run uses CUDA and therefore measures GPU inference latency. The power backend is `nvidia-smi`, so energy is GPU-only. CPU work still exists for Python control, PyTorch dispatch, and data movement orchestration, but CPU energy is not measured by this artifact.

## IV. Power Logging

The measurement script first tries the structured CSV query `-query-gpu=timestamp,power.draw`. On this Windows driver, that field returned [N/A]. The raw `nvidia-smi`

TABLE I  
First PC measurement environment.

| Component          | Setting                            |
|--------------------|------------------------------------|
| System             | Windows 10, AMD64                  |
| GPU                | NVIDIA GeForce RTX 4060            |
| Processor          | Intel64 Family 6 Model 191         |
| Python / PyTorch   | 3.11.9 / 2.11.0+cu128              |
| torchvision / timm | 0.26.0+cu128 / 1.0.27              |
| Device             | CUDA, float32, batch 1             |
| Input              | $1 \times 3 \times 224 \times 224$ |
| Power backend      | nvidia-smi, GPU-only               |
| Window protocol    | 10 windows/model, 10 s/window      |
| Warmup             | 10 inferences before each window   |
| Order              | randomized, seed 20260514          |

TABLE II  
Measurement validity checks in the PC GPU artifact.

| Risk                  | Check or mitigation                                 |
|-----------------------|-----------------------------------------------------|
| Startup overhead      | discard 10 warmup inferences before each window     |
| Order effects         | randomize model order with recorded seed            |
| Power parser failure  | retain raw nvidia-smi output for every window       |
| Missing power samples | require nonzero parsed samples before energy claims |
| Boundary confusion    | label energy as GPU-only, not CPU or wall power     |
| Batch ambiguity       | report batch size and per-image metrics separately  |

-q -d POWER output did include a rolling Power Samples average, so the script falls back to parsing that value while retaining the raw text. This makes the summary auditable: each trial row points to a raw power log, and every completed RTX 4060 window has nonzero parsed power samples.

The resulting energy value is computed as:

$$E_{\text{gpu,inf}} = \frac{\bar{P}_{\text{gpu}} \Delta t}{N}, \quad (1)$$

where  $\bar{P}_{\text{gpu}}$  is mean sampled GPU board power,  $\Delta t$  is the measured window duration, and  $N$  is the number of completed inferences. This is not wall-plug energy and not CPU package energy.

Table II lists the validity checks used in the artifact. These checks are deliberately simple: the goal is to make every summary value traceable back to raw logs and to prevent GPU-only estimates from being read as whole-system measurements.

## V. Results

Table III reports the RTX 4060 measurement summary. All five models have ten parsed energy windows.

The first system already shows why PC measurements should be collected rather than inferred from model size or FLOPs. ResNet-18 has the largest FLOP count in this set,

TABLE III  
RTX 4060 CUDA inference results, 10 windows per model. Energy is GPU-only from nvidia-smi power samples.

| Model         | FLOPs (G) | Energy (J) | 95% CI (J)    | Lat. (ms) | Power (W) |
|---------------|-----------|------------|---------------|-----------|-----------|
| MobileNetV3-S | 0.113     | 0.236      | [0.232,0.241] | 4.31      | 54.91     |
| MobileNetV2   | 0.602     | 0.248      | [0.245,0.252] | 4.38      | 56.67     |
| ResNet-18     | 3.628     | 0.165      | [0.162,0.167] | 2.02      | 81.45     |
| Tiny-ViT-5M   | 2.331     | 0.415      | [0.410,0.419] | 6.95      | 59.68     |
| EffNet-B0     | 0.772     | 0.383      | [0.379,0.388] | 6.80      | 56.35     |

TABLE IV  
Univariate predictors of RTX 4060 GPU-only energy across the five batch-1 model means.

| Predictor              | $R^2$ |
|------------------------|-------|
| Latency (ms/inference) | 0.963 |
| FLOPs (G)              | 0.038 |
| Parameters (M)         | 0.020 |

but it is the fastest and lowest-energy model on the RTX 4060. EfficientNet-B0 has far fewer FLOPs than ResNet-18 but uses over twice the GPU-only energy per inference in this runtime. Across the five model means, latency has  $R^2 = 0.963$  against GPU-only energy; FLOPs has  $R^2 = 0.038$ .

## VI. Batch-Size Sweep

GPU inference efficiency depends strongly on batching, so we ran a second RTX 4060 sweep at batch sizes 1, 4, 8, 16, and 32. This sweep uses the same five models and power backend, but shorter windows: five 5-second windows per model per batch size. The script reports both per-forward metrics and per-image metrics. For batch-size comparisons, per-image energy is the more meaningful quantity:

$$E_{\text{gpu,img}} = \frac{\bar{P}_{\text{gpu}} \Delta t}{NB}, \quad (2)$$

where  $B$  is batch size.

Table V summarizes the best observed batch size for each model. The improvement column compares batch size 1 against the best per-image energy observed in the sweep.

The best batch size is not the same for every model. MobileNetV3-Small continues improving through batch 32, while MobileNetV2 and Tiny-ViT-5M reach their lowest per-image energy at batch 8. ResNet-18 and EfficientNet-B0 reach their best per-image energy at batch 16. This matters for publication-quality benchmarking: a single batch-1 table measures low-latency serving behavior, but it does not characterize throughput-oriented GPU efficiency. Future PC systems should therefore report batch size explicitly and, where possible, include both batch-1 and throughput-oriented measurements.

## VII. Discussion

The RTX 4060 results show two separate ways that FLOPs can mislead. First, at batch size 1, ResNet-18

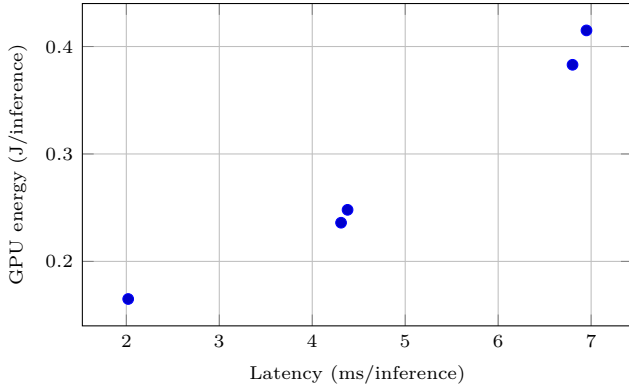


Fig. 1. RTX 4060 GPU-only energy versus latency for batch-1 CUDA inference. Latency explains most between-model energy variation in this run.

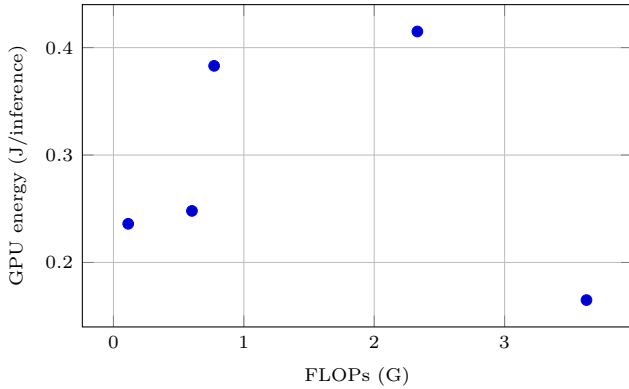


Fig. 2. RTX 4060 GPU-only energy versus FLOPs. FLOPs ranks ResNet-18 as the largest model by arithmetic count, but it is the lowest-energy model here.

has the highest FLOP count but the lowest measured GPU-only energy because its CUDA execution is much faster than the other models. Higher average board power does not imply higher energy when latency falls enough to shorten the measured window work per inference. Second, batching changes the denominator of the energy calculation. A model can draw more power at a larger batch size while still using less energy per image because throughput increases more than power.

This distinction is important for HPEC-style benchmarking. Edge and embedded deployments may operate in latency-oriented single-request mode, while offline or buffered deployments may operate in throughput mode. Reporting only one batch size hides this choice. The current artifact therefore treats batch size as part of the measurement condition, not a minor implementation detail.

The current data also suggests that the most useful cross-system comparison is not absolute joules across dissimilar machines, but whether model rankings and best batch sizes remain stable under each system’s own power boundary. If a second PC shows a different ranking, that

TABLE V  
Best RTX 4060 batch size by GPU-only energy per image.

| Model         | Best $B$ | E/img (J) | Images/s | Gain ( $\times$ ) |
|---------------|----------|-----------|----------|-------------------|
| MobileNetV3-S | 32       | 0.0127    | 7495     | 18.6              |
| MobileNetV2   | 8        | 0.0451    | 2012     | 5.4               |
| ResNet-18     | 16       | 0.0464    | 2114     | 3.3               |
| Tiny-ViT-5M   | 8        | 0.1039    | 958      | 3.9               |
| EffNet-B0     | 16       | 0.0668    | 1419     | 5.8               |

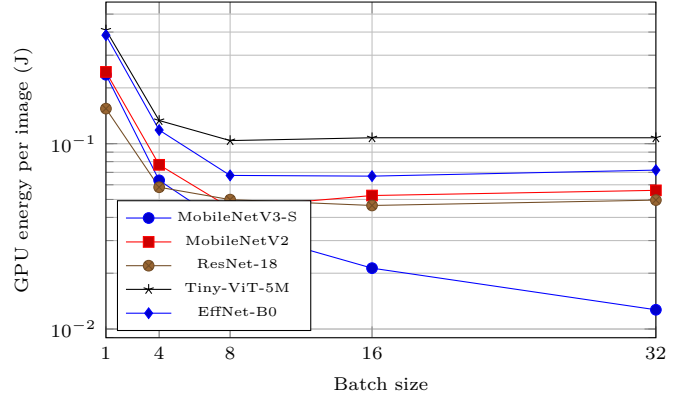


Fig. 3. Batching reduces GPU energy per image, but the best batch size is model-dependent on this RTX 4060 run.

is not a failure of the benchmark; it is evidence that deployment measurements are necessary.

## VIII. Protocol for Additional PC Systems

The next step toward an HPEC-ready multi-system paper is adding at least one more non-Mac machine with the same artifact format. A second Windows PC should run the batch-1 command first:

```
python scripts\measure_pc_inference.py --suite paper
--device cuda --power-backend nvidia-smi
--windows 10 --window-seconds 10 --warmups 10
```

If time permits, it should then run the same batch sweep used here:

```
python scripts\measure_pc_inference.py
--suite paper --device cuda
--power-backend nvidia-smi
--windows 5 --window-seconds 5
--warmups 10 --batch-size B
--output-dir pc_batch_B
```

for  $B \in \{1, 4, 8, 16, 32\}$ , lowering the maximum batch size only if the GPU runs out of memory.

Each contributed system should include the complete output folder, not just the summary CSV. The required files are the trial CSV, summary CSV, environment JSON, and the full raw\_power/ directory. If parsed\_power\_samples is zero for any row, that run should be treated as latency-only until the raw log parser or power backend is fixed.

TABLE VI  
Minimum metadata required for each added PC system.

| Field                 | Reason                                                          |
|-----------------------|-----------------------------------------------------------------|
| GPU model and driver  | power telemetry support changes by device and driver            |
| CPU model             | CPU dispatch overhead is outside GPU energy but affects runtime |
| PyTorch/CUDA versions | kernel selection can change latency and power                   |
| Power backend         | distinguishes GPU-only, CPU package, and wall power             |
| Batch sizes           | separates low-latency serving from throughput operation         |
| Raw logs              | allows parser failures and missing samples to be audited        |

### IX. Expansion Plan

This paper should grow by adding more non-Mac systems as separate rows or subsections, not by mixing them into the Apple-Silicon paper. The next systems should reuse the same windowed protocol where possible:

- fixed model set and input shape,
- explicit batch size or batch sweep,
- recorded environment metadata,
- randomized model order,
- raw power logs retained,
- explicit power boundary: GPU board, CPU package, or wall power.

If a future Windows system cannot expose GPU power through `nvidia-smi`, its rows should be marked latency-only unless an external or vendor-specific power source is added. If a CPU run is needed on Windows, the current script does not provide direct CPU package energy; it would need an added backend or external meter.

### X. Limitations

The current PC paper has one measured system. The RTX 4060 values are GPU-only board-energy estimates from `nvidia-smi`, not total system energy. They exclude CPU package energy, memory outside the GPU board, storage, display, and PSU losses. The five-model regression is descriptive rather than universal. These limitations are acceptable for a first artifact because the raw logs and metadata are retained, but broader claims require additional PC systems and power backends. The batch sweep uses shorter windows and fewer repeats than the main batch-1 run, so it should be read as a throughput trend rather than as the final uncertainty estimate for each batch size.

### XI. Artifact Availability

The measurement artifacts are in the repository under `pc_measurements/` and in the archive `rtx4060_measurements.zip`. Key files are `pc_inference_trials.csv`, `pc_inference_summary.csv`,

`pc_measurement_environment.json`, and the `raw_power/` directory. The batch-size sweep is under `pc_measurements_rtx4060_batch/`.

### XII. Conclusion

The RTX 4060 run measured CUDA latency and GPU-only energy, not CPU energy. Within that boundary, latency was a much stronger predictor of energy than FLOPs. The batch sweep further shows that GPU energy per image depends heavily on batching, with model-specific optima across batch sizes 8, 16, and 32. The result supports a separate PC measurement paper that can expand across additional Windows and non-Mac systems without entangling those results with the MacBook CPU-only study.

### References

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2018.
- [3] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. International Conference on Machine Learning, 2019.
- [4] V. J. Reddi et al., “MLPerf Inference Benchmark,” in Proc. ACM/IEEE International Symposium on Computer Architecture, 2020.
- [5] NVIDIA, “NVIDIA System Management Interface,” available: <https://developer.nvidia.com/system-management-interface>.